



1. Problem Title & Link

Title: LeetCode 922 – Sort Array By Parity II

Link: <https://leetcode.com/problems/sort-array-by-parity-ii/>

2. Problem Statement (Short Summary)

You are given an integer array `nums` where:

- Half of the integers are **even**.
- Half of the integers are **odd**.

Rearrange the array so that:

- Every **even index** contains an **even number**.
- Every **odd index** contains an **odd number**.

Return any valid arrangement.

3. Examples (Input → Output)

Example 1

Input

`nums = [4,2,5,7]`

Output

`[4,5,2,7]`

Explanation

Index 0 → 4 (Even)

Index 1 → 5 (Odd)

Index 2 → 2 (Even)

Index 3 → 7 (Odd)

Example 2

Input

`nums = [2,3]`

Output

`[2,3]`

4. Constraints

- $2 \leq \text{nums.length} \leq 2 \times 10^4$
- `nums.length` is even.



- Half of the elements are even.
- Half of the elements are odd.

5. Core Concept (Pattern / Topic)

- Two Pointers
- In-place Swapping
- Array Manipulation

6. Thought Process (Step-by-Step Explanation)

Brute Force

Create two separate lists:

- One for even numbers.
- One for odd numbers.

Fill the answer array alternately.

Time Complexity

$O(n)$

Space Complexity

$O(n)$

Optimized Thinking

Instead of creating extra arrays,

observe:

- Even indices should contain even numbers.
- Odd indices should contain odd numbers.

Maintain two pointers:

even = 0

odd = 1

- even moves over even indices.
- odd moves over odd indices.

Whenever both pointers point to incorrect values,
swap them.

Intuition

Example

[4,2,5,7]

Pointers



even → index 2 (contains 5 ❌)

odd → index 1 (contains 2 ❌)

Swap

[4,5,2,7]

Done.

Why This Works

Every swap fixes **two incorrect positions** simultaneously:

- The misplaced odd number goes to an odd index.
- The misplaced even number goes to an even index.

Since every element is visited at most once, the solution is linear.

7. Visual / Intuition Diagram (ASCII)

Initial

Index

0 1 2 3

4 2 5 7

↑ ↑

odd even

Swap

0 1 2 3

4 5 2 7

Correct ✓

8. Pseudocode (Language Independent)

```
even = 0
```

```
odd = 1
```

```
while even < n and odd < n
```

```
    move even until wrong
```

```
    move odd until wrong
```



```
        if both valid

            swap

    return nums
```

9. Code Implementation

Python

```
class Solution:
    def sortArrayByParityII(self, nums: list[int]) -> list[int]:

        even = 0
        odd = 1
        n = len(nums)

        while even < n and odd < n:

            while even < n and nums[even] % 2 == 0:
                even += 2

            while odd < n and nums[odd] % 2 == 1:
                odd += 2

            if even < n and odd < n:
                nums[even], nums[odd] = nums[odd], nums[even]

        return nums
```

Java

```
class Solution {

    public int[] sortArrayByParityII(int[] nums) {

        int even = 0;
        int odd = 1;
        int n = nums.length;

        while (even < n && odd < n) {

            while (even < n && nums[even] % 2 == 0)
                even += 2;

            while (odd < n && nums[odd] % 2 == 1)
                odd += 2;

            if (even < n && odd < n) {
```



```

        int temp = nums[even];
        nums[even] = nums[odd];
        nums[odd] = temp;
    }
}

return nums;
}
}

```

10. Time & Space Complexity

Time Complexity

$O(n)$

Each pointer moves through its respective indices only once.

Space Complexity

$O(1)$

The rearrangement is performed in-place.

11. Common Mistakes / Edge Cases

- Incrementing pointers by 1 instead of 2.
- Swapping before locating misplaced elements.
- Forgetting the boundary checks ($\text{even} < n$, $\text{odd} < n$).
- Assuming even and odd counts are unequal (the problem guarantees they are equal).

12. Detailed Dry Run

Example

nums = [4,2,5,7]

Even Pointer	Odd Pointer	Array	Action
0	1	[4,2,5,7]	0 is correct
2	1	[4,2,5,7]	Both incorrect
2	1	[4,5,2,7]	Swap
4	3	Done	Finish

Output

[4,5,2,7]

13. Common Use Cases (Real-Life / Interview)



- Position-based rearrangement.
- Two-pointer partitioning.
- In-place array transformations.
- Alternating placement problems.
- Scheduling with alternating constraints.

14. Common Traps (Important!)

- Using $\%2==0$ on odd pointer.
- Moving pointers after swapping incorrectly.
- Using extra arrays when in-place is expected.
- Forgetting pointers move by two positions.

15. Builds To (Related LeetCode Problems)

- LC 905 – Sort Array By Parity
- LC 75 – Sort Colors
- LC 283 – Move Zeroes
- LC 27 – Remove Element
- LC 2149 – Rearrange Array Elements by Sign

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Extra Even/Odd Arrays	$O(n)$	$O(n)$	Simple but uses extra memory
Two-Pointer In-place	$O(n)$	$O(1)$	Best interview solution
Sorting	$O(n \log n)$	$O(1)$	Unnecessary and slower

17. Why This Solution Works (Short Intuition)

The even pointer searches only even indices for misplaced odd numbers, while the odd pointer searches only odd indices for misplaced even numbers. Swapping these two misplaced elements corrects both positions in one operation, resulting in a linear-time, constant-space solution.

18. Variations / Follow-Up Questions

1. Solve when the number of even and odd elements is not equal.
2. Return a new array instead of modifying in-place.
3. Generalize for alternating positive and negative numbers (LC 2149).



4. Rearrange so that odd indices come first instead of even indices.
5. Extend the solution for three categories (e.g., even, odd, and prime indices).
- 6.